CDT204
Computer Architecture

- Afshin Ameri

Outline

01 • Course Overview

02 • Introduction to Computer Architecture

03 • Performance

.Course Overview

What is covered in the course?

Covered Topics

- Performance
- Assembly programming
- Pipelining
- Memory hierarchies
- Microprogramming
- Multiprocessing

Example: Code Efficiency

OPTION A

```
int sum_mat_ji(int mat[N][N])
{
   int sum = 0;
   for (int j=0;j<N;j++)
      for (int i=0;i<N;i++)
         sum += mat[i][j];
   return sum;
}
```

Execution time with N=10000
1.264 sec

OPTION B

```
int sum_mat_ij(int mat[N][N])
{
   int sum = 0;
   for (int i=0;i<N;i++)
      for (int j=0;j<N;j++)
         sum += mat[i][j];
   return sum;
}
```

Execution time with N=10000
0.052 sec
✓ 24 times faster

Example: Code Efficiency

OPTION A

```
void abs_arr_1(int arr[N])
{
   for (int i=0;i<N;i++)
      if (arr[i]<0)
         arr[i]=-arr[i];
}
```

Execution time N=100000000
0.42 sec

OPTION B

```
void abs_arr_1(int arr[N])
{
   for (int i=0;i<N;i++)
   {
      int sgn=arr[i]>>31;
      arr[i]=(arr[i]^sgn)-sgn;
   }
}
```

Execution time N=100000000
0.11 sec
✓ 4 times faster

Example: Assembly Code

```
    lw  $s0, x
    slt $t0, $s0, $zero
    beq $t0, $zero, skip
    sub $s0, $zero, $s0
    sw  $s0, x
skip:
    ...
```

Margin note

- This is an example of an assembly code sequence, computing the absolute value of a variable x.
- Each row is a machine instruction, and this is the language that a CPU understands. We will learn how compilers produce such instruction sequences from a high-level language, and we will also learn how to write assembly code ”by hand”.

Example: Using SIMD

```
float sum_array_simd(float arr[N])
{
    __m128 vec_sum = _mm_setzero_ps();
    for (int i = 0; i < N; i += 4)
    {
        __m128 vec = _mm_load_ps(arr + i);
        vec_sum = _mm_add_ps(vec_sum, vec);
    }
    __declspec(align(16)) float vec_sum_arr[4];
    _mm_store_ps(vec_sum_arr, vec_sum);
    return vec_sum_arr[0] + vec_sum_arr[1]
                + vec_sum_arr[2] + vec_sum_arr[3];
}
```

Margin note

- A function that sums together the elements of an array using SIMD instructions, which are available on most modern CPU:s. Using these instructions, up to eight arithmetic operations can be performed in parallel, resulting in significant speed-ups.

Example: Using CUDA

```
__global__ void sharedMemoryMax(float* d_out, float* d_in, int dataSize){
   extern __shared__ float shared[];
   int gid = threadIdx.x + blockDim.x * blockIdx.x;
   int tid = threadIdx.x;
   shared[tid] = d_in[(gid % dataSize)];
   __syncthreads();
   for (unsigned int wSet = blockDim.x / 2; wSet > 0; wSet >>= 1){
      if (tid < wSet)
         shared[tid]=max(shared[tid],shared[tid+wSet]);
      __syncthreads();
   }
   if (tid == 0)
      d_out[blockIdx.x] = shared[0];
}
```

Margin note

- A function that finds the maximum of an array using Cuda. Large arrays can be “*reduced*” by max function very fast using parallel computing

About the Course

- Course Responsible
  - Afshin Ameri
  - afshin.ameri *at* mdu *dot* se
  - 021-101550
- Teachers
  - Jean Malm
  - Gabrielle Capannini

Contact us primarily through email and put CDT204 in title. Include ALL teachers in the email. Use “**reply all**” when responding.

On the Web

This course is using Canvas system. You can find all the information, lecture slides, assignment descriptions, support files and even some cookies there.

Old lecture slides are published. New revisions will be published after the lectures.

We do **NOT** use Canvas for *communication*.

Literature

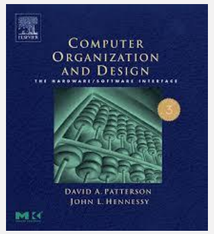
- Course Textbook (available through MDU library as eBook):

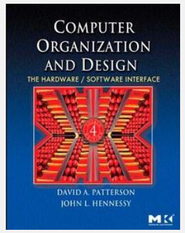
Computer Organization and Design

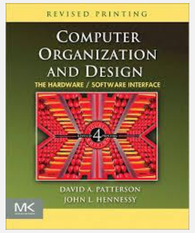
The hardware/software interface

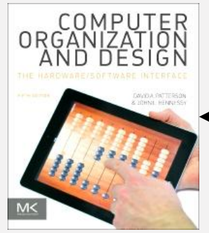
David A. Patterson and John L. Hennessy

Any of 4th Edition, Revised 4th Editions or 5th Edition. Versions of the book that focus on a specific architecture (eg. ARM, x86, etc) are not useful for this course









Examination

- LAB1, 3hp
  - 4 programming assignments
  - Quiz examination
  - Pass the Quiz for the LAB1 grade

- TEN1, 4.5hp
  - Written exam
  - Grade on exam = final grade for course

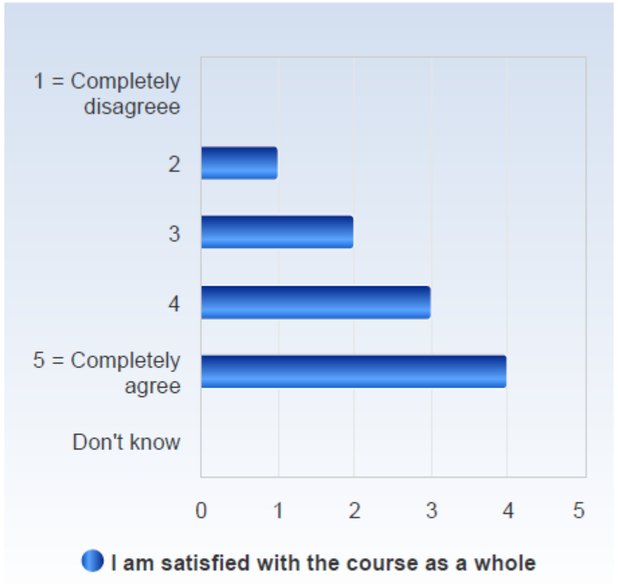
Lab Assignments

- 4 Lab assignments
- When published, work on the assignment before the lab session, so you can use the lab time to ask questions from the assistants.
- You cannot start and finish a lab just by using the lab time.



Course Evaluation 2025

I am satisfied with the course as a whole	Number of responses
1 = Completely disagreee	0 (0,0%)
2	1 (10,0%)
3	2 (20,0%)
4	3 (30,0%)
5 = Completely agree	4 (40,0%)
Don't know	0 (0,0%)
Total	10 (100,0%)



	Mean	Standard Deviation	Coefficient of Variation	Min	Lower Quartile	Median	Upper Quartile	Max
I am satisfied with the course as a whole	4,0	1,1	26,4 %	2,0	3,5	4,0	5,0	5,0

Course Evaluation 2025

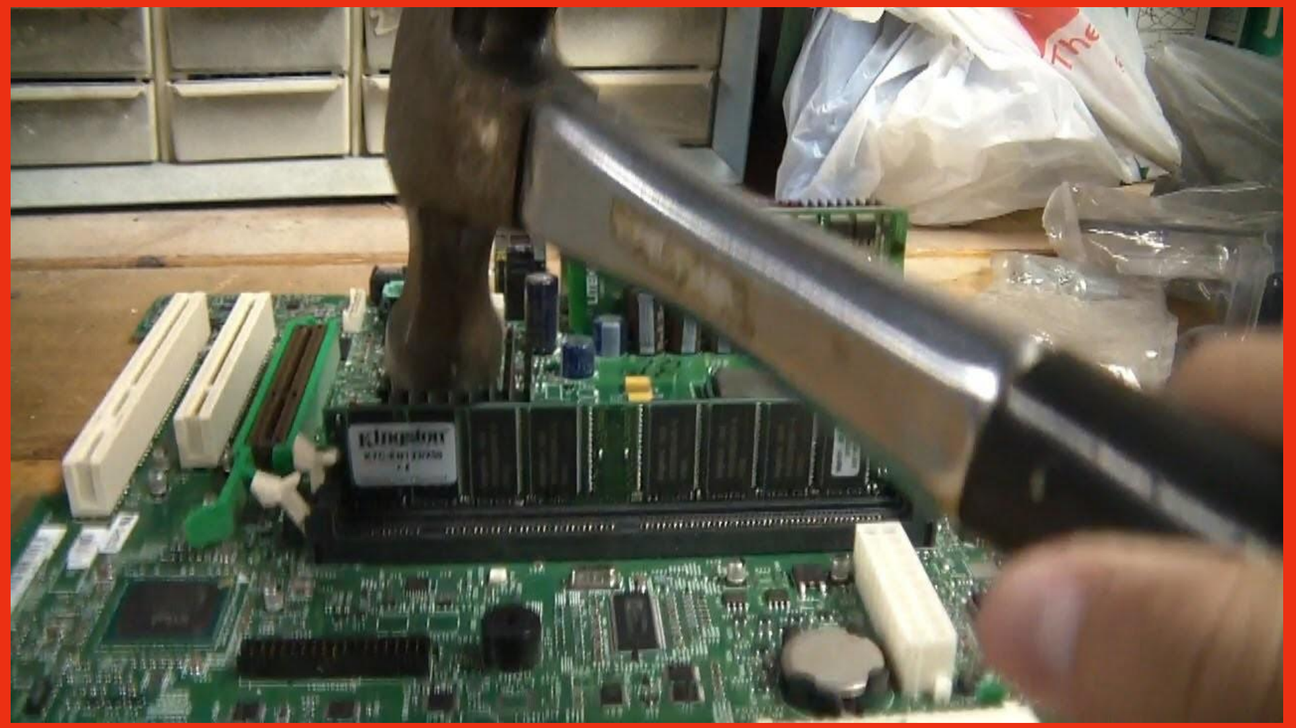
Text comments from evaluation:

- Tough at times, a lot of information to unpack most of the lectures. Maybe empathize that reading the course book before each lecture to be familiar with the content even more, cannot stress how much this helped me.

- Bra och intressant kurs. Skulle 100% rekommendera till andra. Nästan lite synd att den inte låg tidigare i programmet. Speciellt labb 1 hade

- Very fun course that teaches you a lot about internal workings of a processor. Highly recommend!

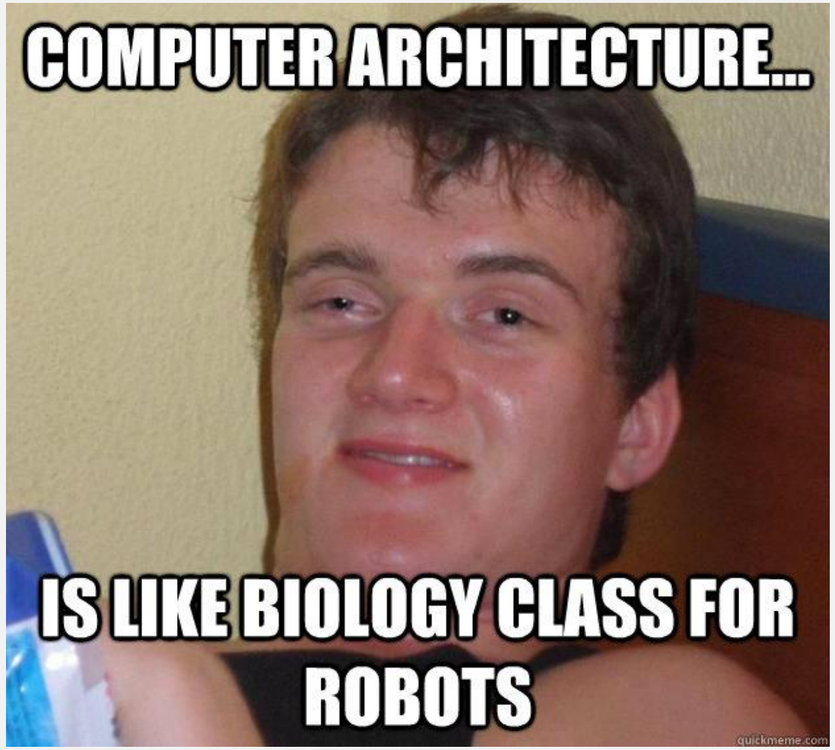
- Crash Course in Computer Architecture



What is Computer Architecture?

Computer Architecture

- What is it?



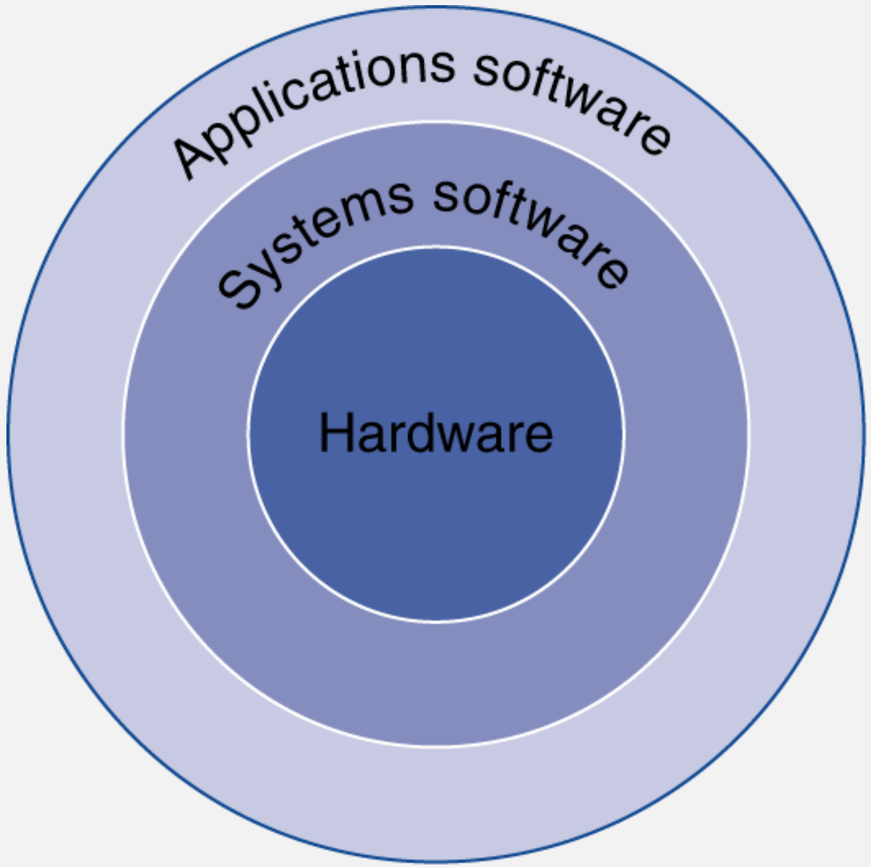
Abstractions

- Deal with complexity
  - Hide lower level details

- Several layers of abstraction:
  - Source code using libraries
  - Assembly code
  - Machine code
  - Implementation

Below your Program

- Application software
- System software
  - Operating system
  - Drivers
- Hardware
  - Processor
  - Memory
  - I/O
  - Etc.



Hardware/Software Interface

- Instruction set architecture (ISA)
  - Available instructions and registers
- Application binary interface
  - The ISA plus system software interface
- Assembly code and machine code

Overview of a Computer



Processor

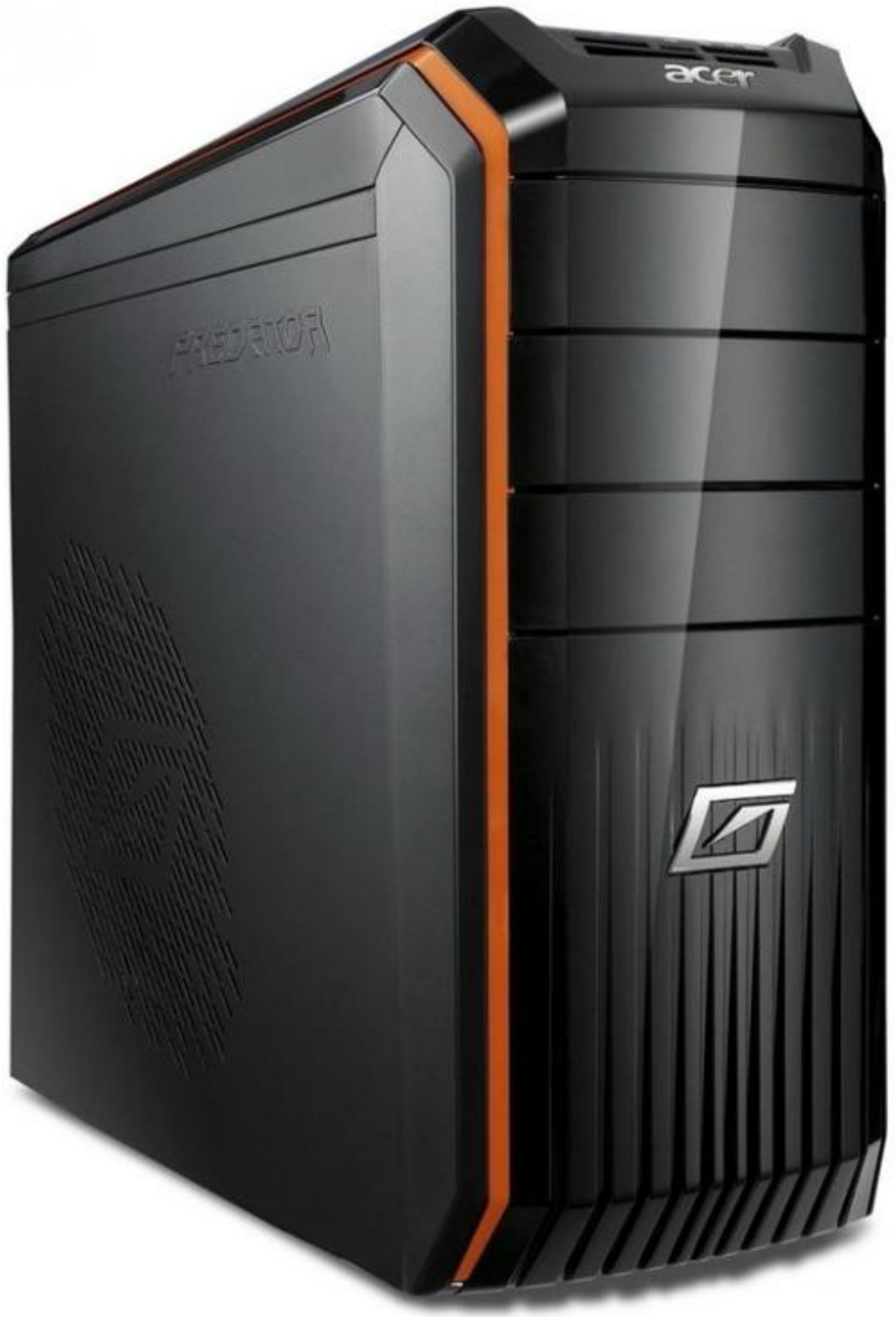
Input/Output

Memory



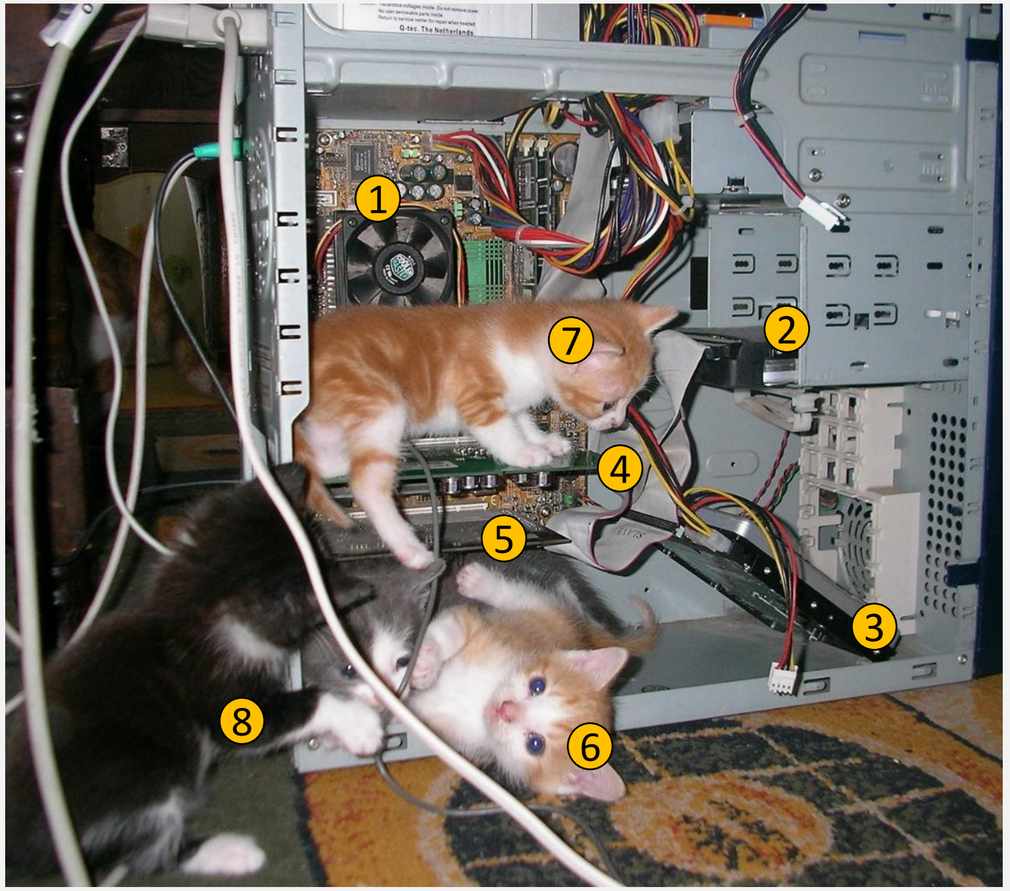
Under the Hood

- The question is if I open the case what is inside?



Under the Hood

1. CPU under the fan
2. Hard disk
3. Another hard disk (45º)
4. Graphic card
5. Some other I/O device
6. Cat
7. Another cat
8. Black cat



Under the Hood



Classes of Computers

- Desktop computers
  - General purpose, variety of software
  - Subject to cost/performance tradeoff
- Server computers
  - Network-based
  - High capacity, performance, reliability
  - Range from small servers to building sized
- Embedded computers
  - Hidden as components of systems
  - Power/performance/cost constraints

Advances in Information Technology

- Moore’s law

- Applications made feasible:
  - Embedded computers, e.g. in cars
  - Cellphones, smart phones, tablets
  - WWW, search engines
  - Singularity? (someday in future)

Technology trends

Year	Technology	Relative performance/cost
1951	Vacuum tube	1
1965	Transistor	35
1975	Integrated circuit (IC)	900
1995	Very large scale IC (VLSI)	2,400,000
2005	Ultra large scale IC	6,200,000,000

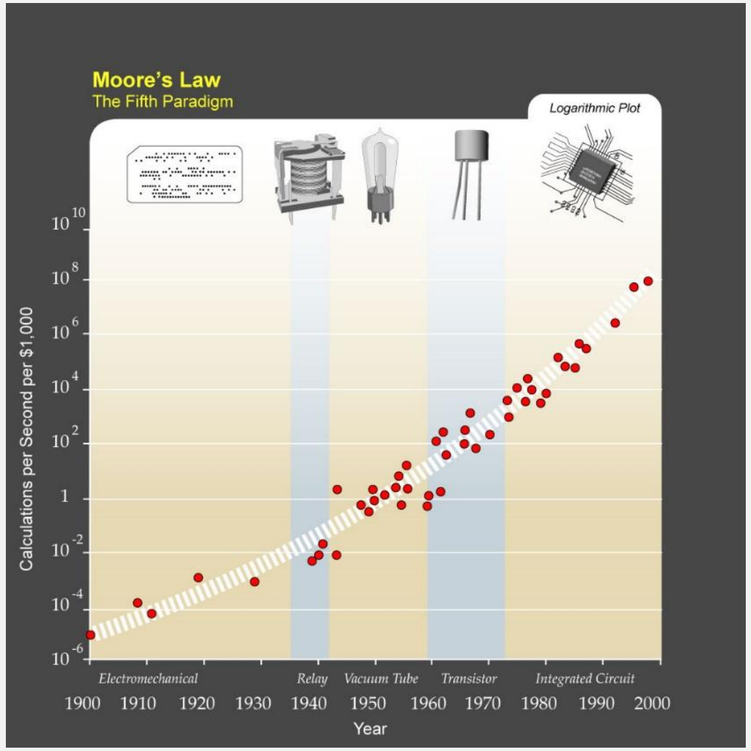
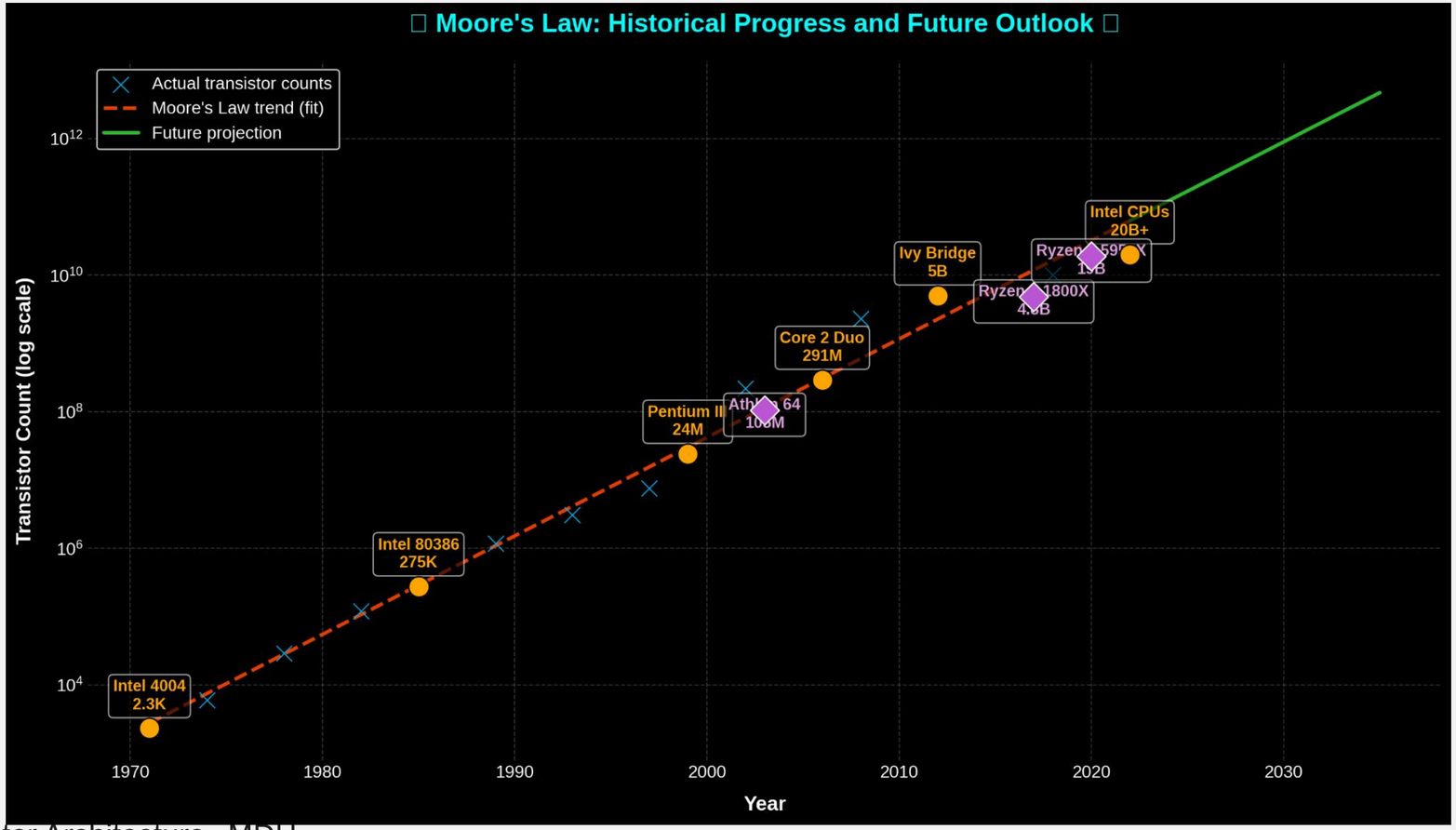


Image Coutesy of Ray Kurzweil

Technology trends



Advances in Information Technology

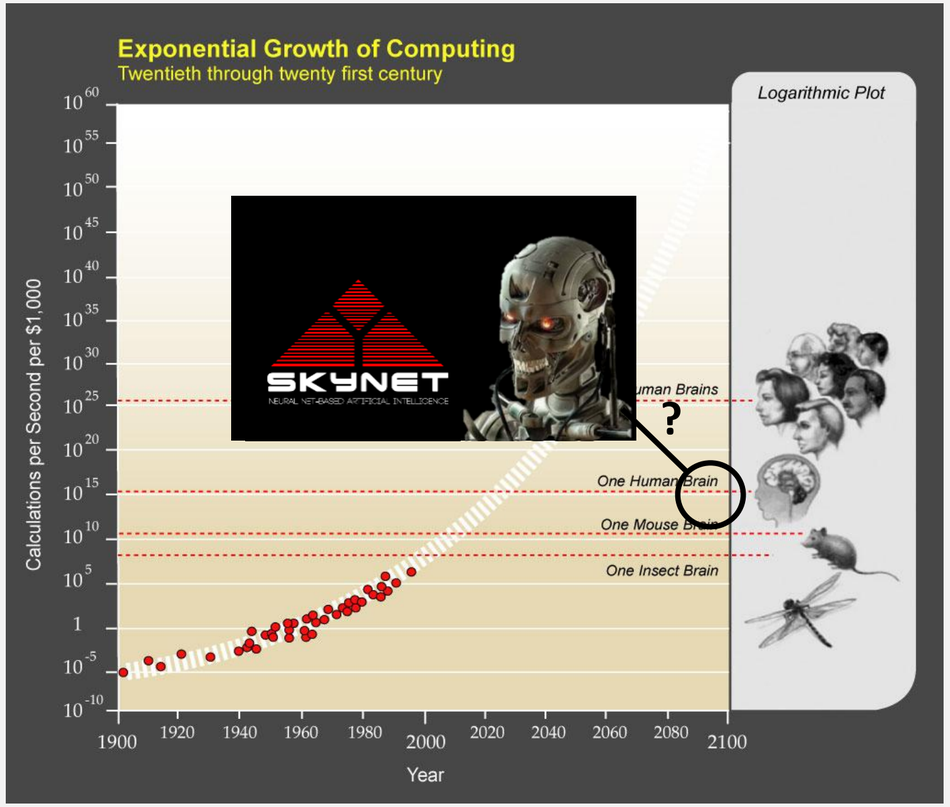
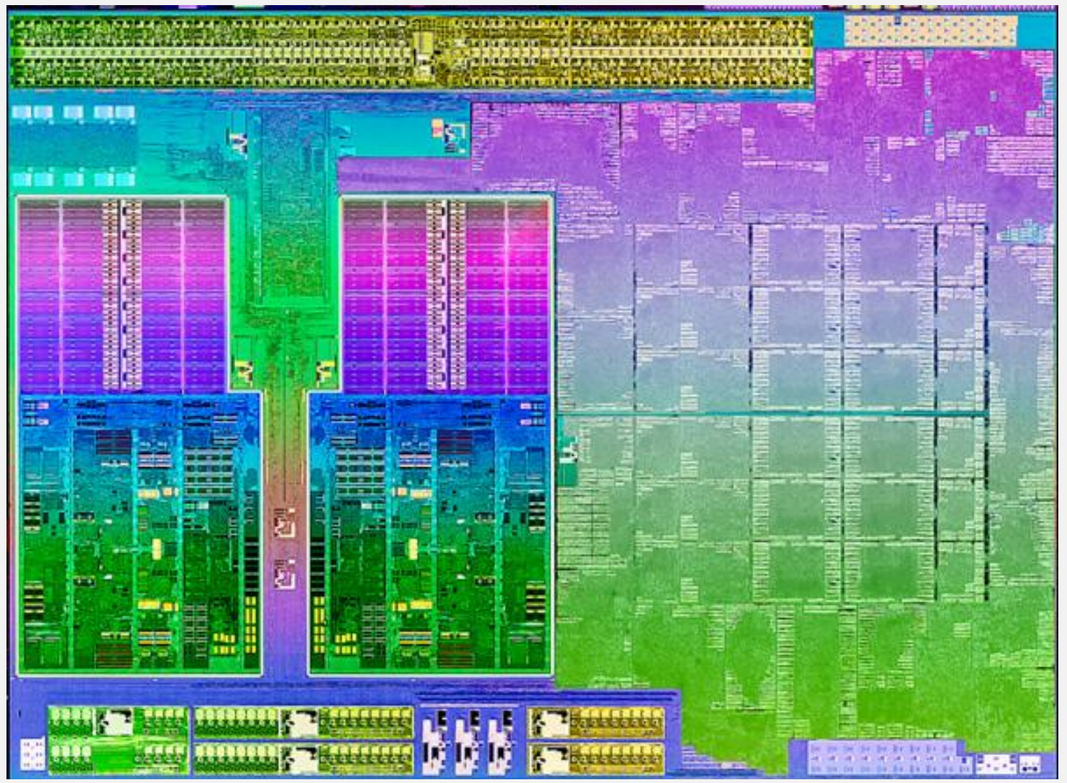


Image Coutesy of Ray Kurzweil

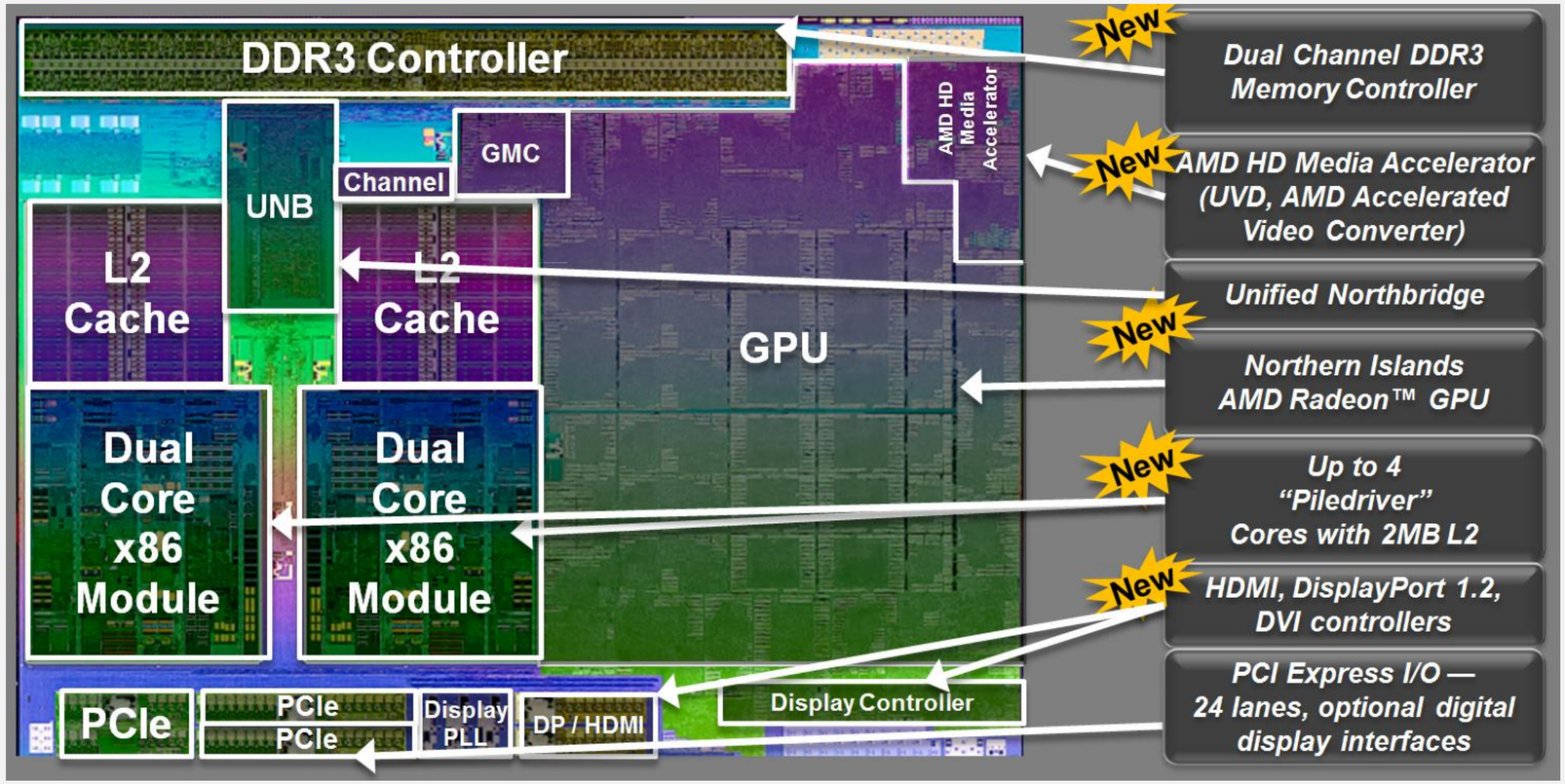
Integrated Circuits

- “IC”, “Microchip” or “Chip”
- Can contain billions of transistors
- Advantages:
  - High performance
  - Low cost
  - Reliable

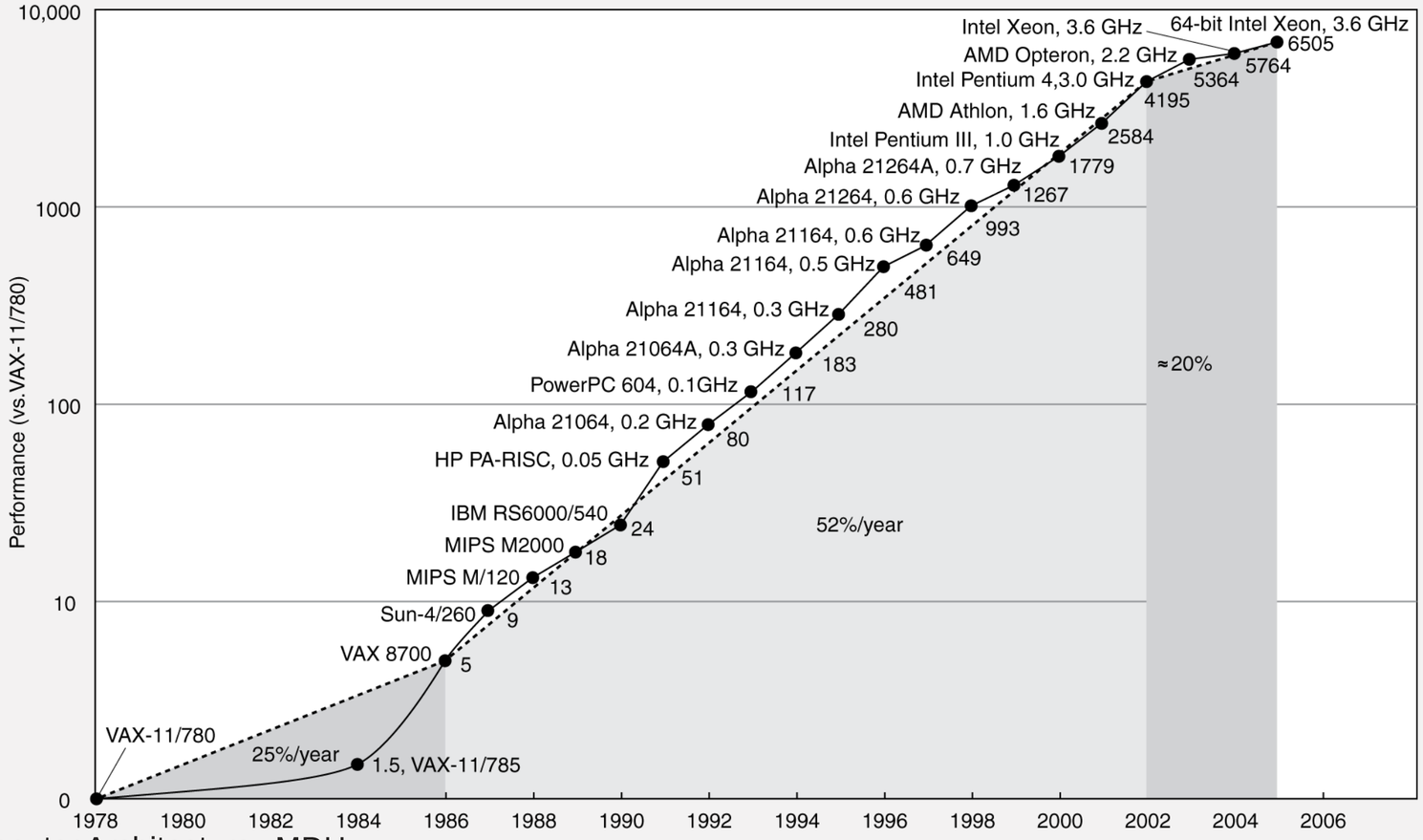
Integrated Circuits



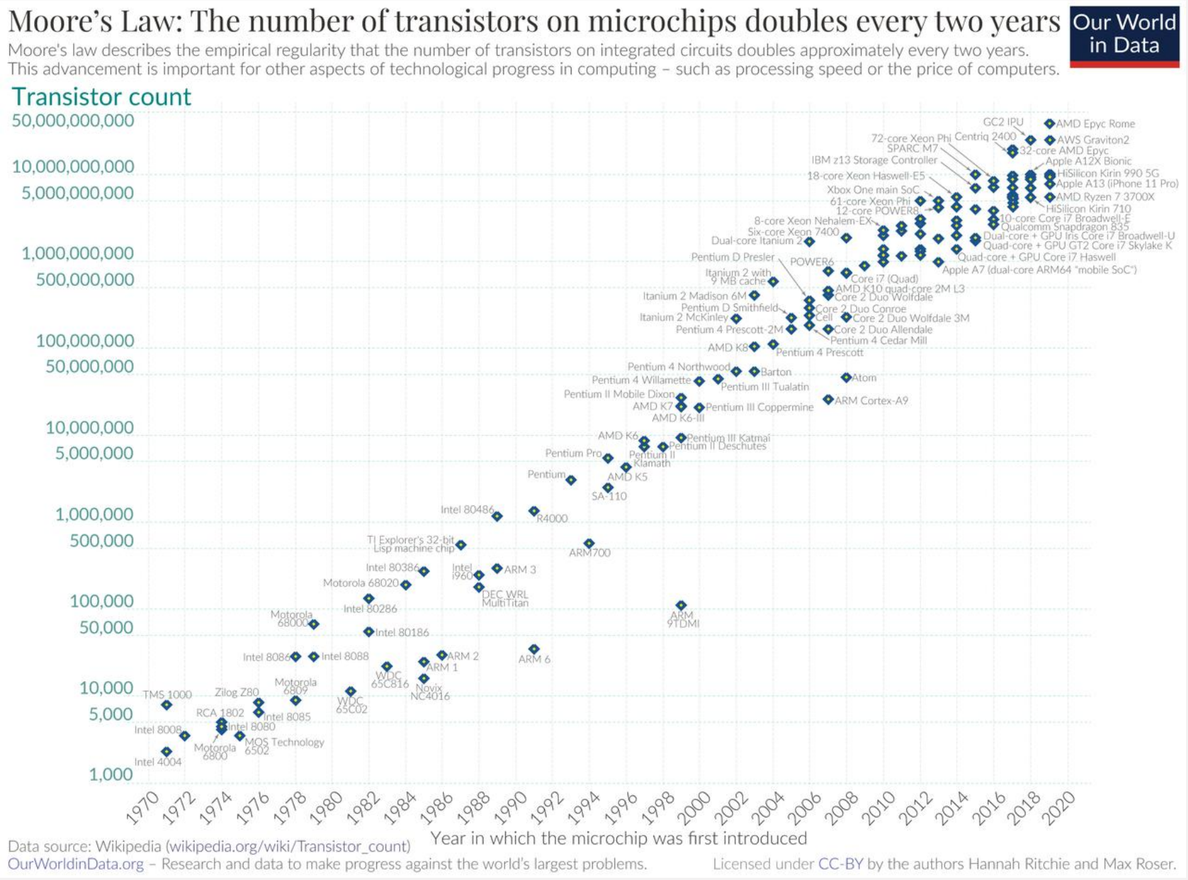
Integrated Circuits



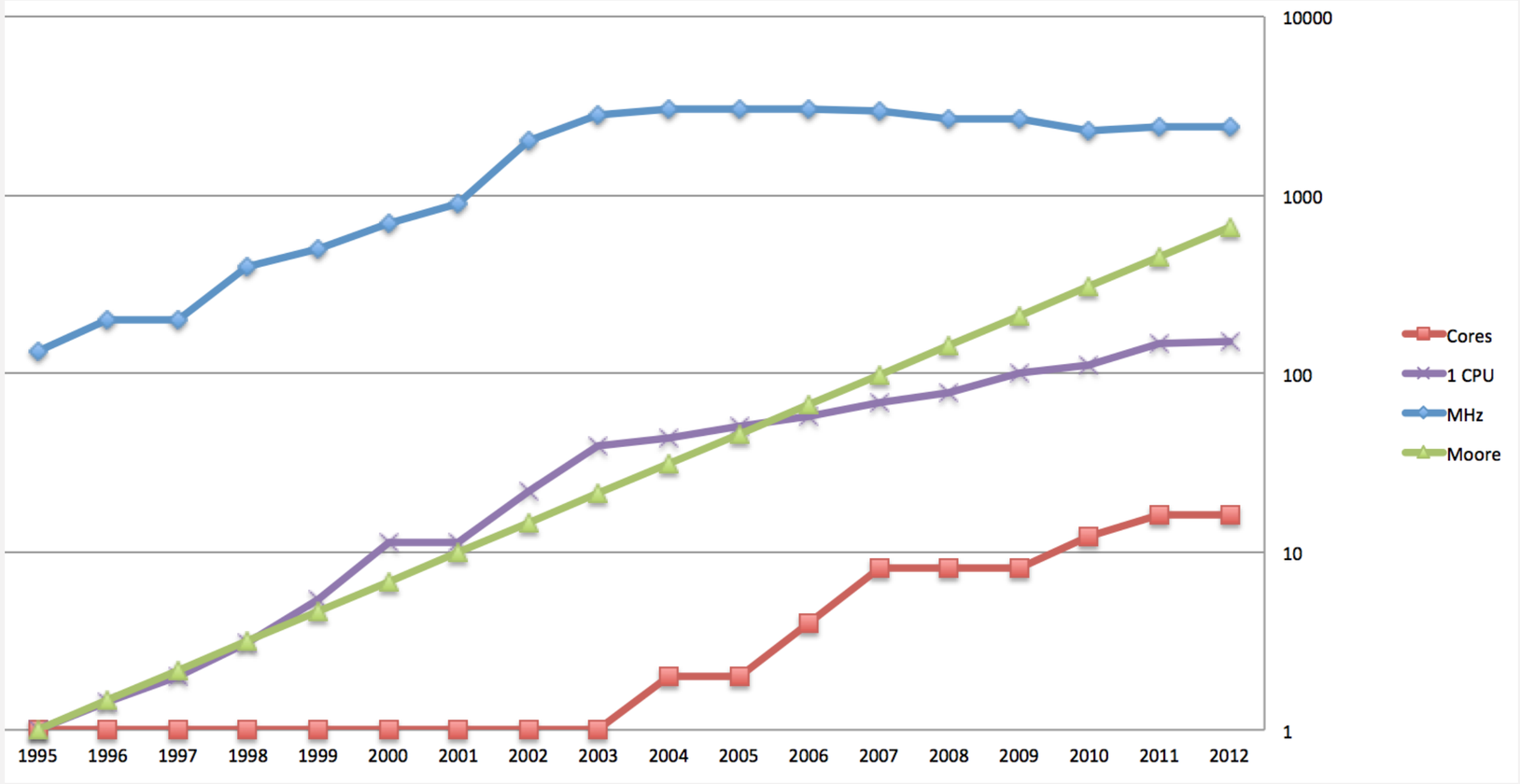
Uniprocessor Performance



Uniprocessor Performance



Performance



Parallel Computing

- Do more work at the same time
- Different types:
  - Bit-level parallelism
  - Instruction-level parallelism
  - Data parallelism
  - Task parallelism

Multiprocessors

- Multiple processors
- Multiple cores on a single die

- Challenges:
  - Explicit parallel programming
  - Load balancing
  - Communication and synchronization

.Performance

What is it and why it is important?

Performance

- What is performance?
  - Response time for single task?
  - Throughput?
- What determines performance?
  - Chosen algorithm
  - CPU clock frequency
  - Cache memory size/speed
  - Memory access patterns
  - …

Program Performance

- ❶ Algorithm
- ❷ Programming language
- ❸ Compiler
- ❹ Processor
- ❺ Memory system
- ❻ I/O system (HW and OS)

Program Performance

- ❶ Algorithm
- ❷ Programming language
- ❸ Compiler
- ❹ Processor
- ❺ Memory system
- ❻ I/O system (HW and OS)

Determines:

- Number of source-level statements
- Number of I/O operations

Program Performance

- ❶ Algorithm
- ❷ Programming language
- ❸ Compiler
- ❹ Processor
- ❺ Memory system
- ❻ I/O system (HW and OS)

Determine number of computer instructions for each source-level statement

Program Performance

- ❶ Algorithm
- ❷ Programming language
- ❸ Compiler
- ❹ Processor
- ❺ Memory system
- ❻ I/O system (HW and OS)

Determine:

- How fast the instructions can be executed
- How fast the information is available from the memory

Program Performance

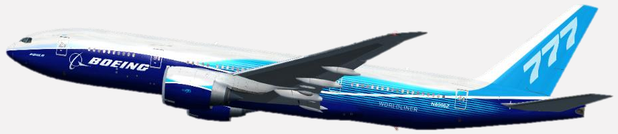
- ❶ Algorithm
- ❷ Programming language
- ❸ Compiler
- ❹ Processor
- ❺ Memory system
- ❻ I/O system (HW and OS)

Determines how fast the I/O operations can be executed

Performance

Which one has a better performance?

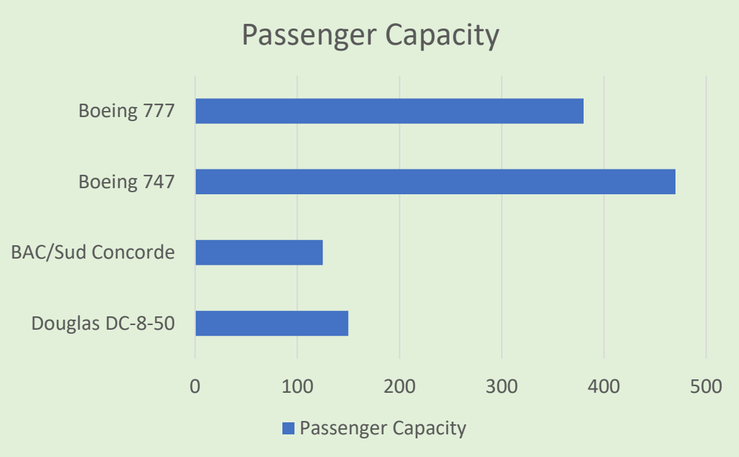


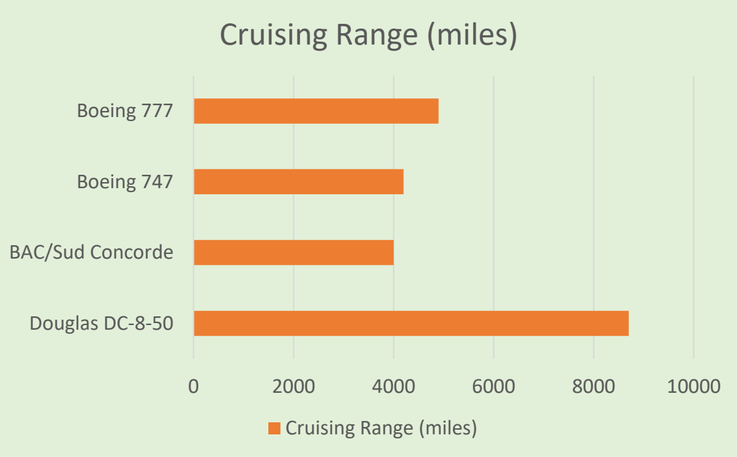


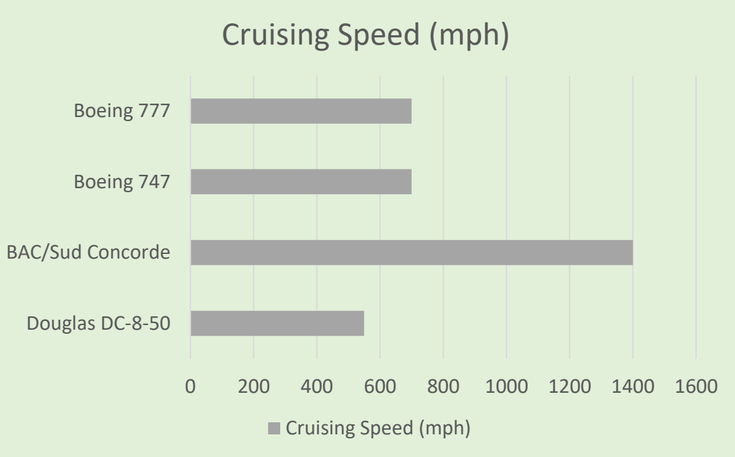




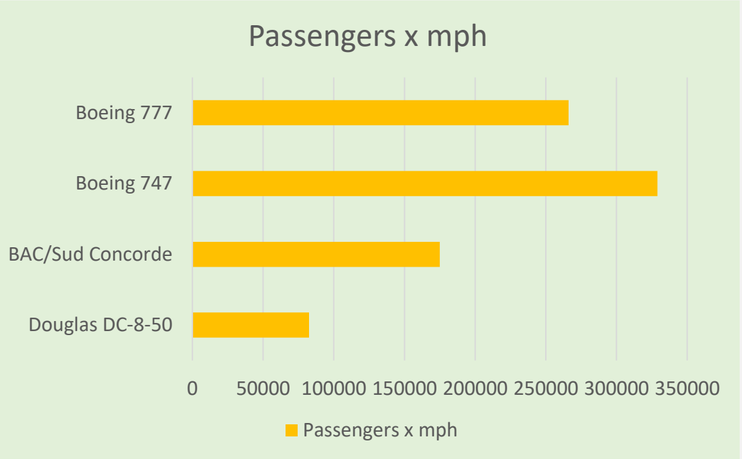
Performance

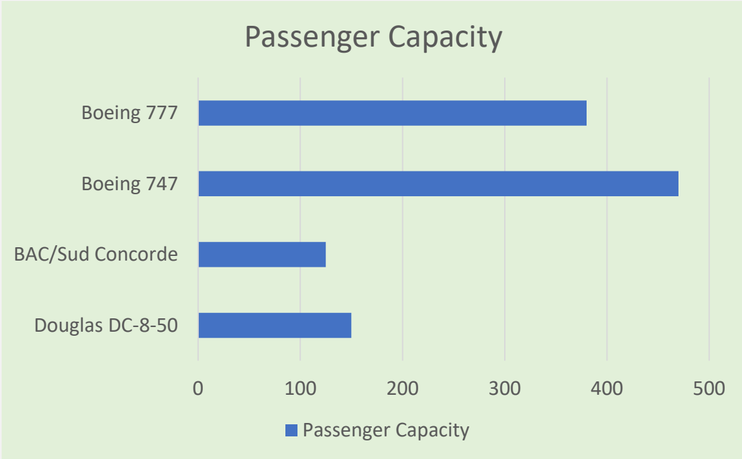


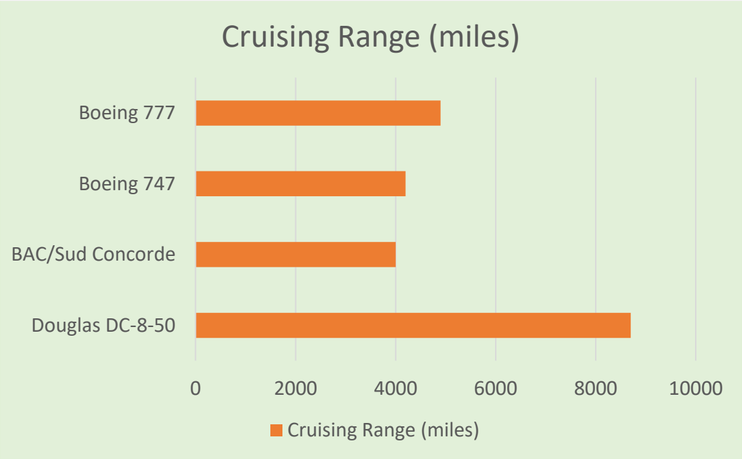


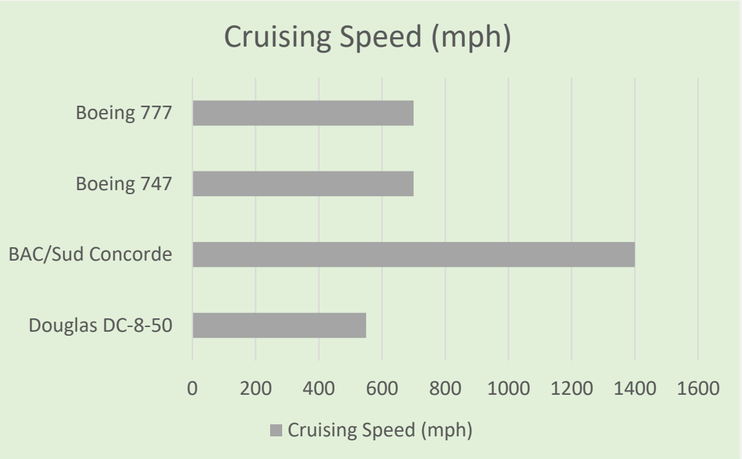


Performance









Relative Performance

- Let $performance = \frac{1}{Execution\ time}$
- *X* is *n* times faster then *Y* if:

$$\frac{Performance_X}{Performance_Y} = n \quad \Rightarrow \quad \frac{Execution\ time_Y}{Execution\ time_X} = n$$

Execution Time

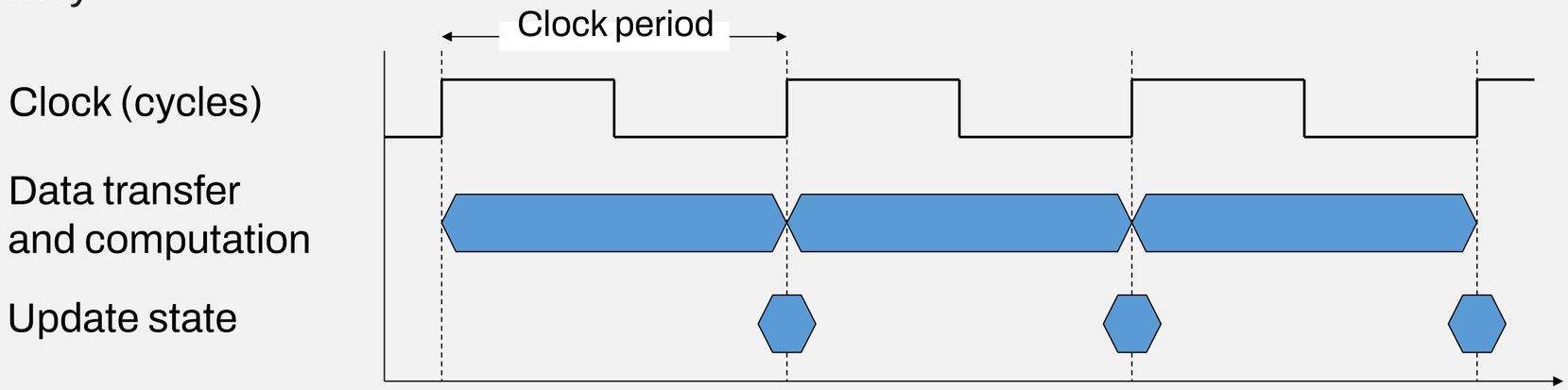
- *Elapsed time* includes
  - The time our own task takes
  - The time other related tasks take
  - I/O
- *CPU time* includes
  - The time our own task takes

Performance Parameters

- Instruction count:
  - Number of “work items”
- Cycles per instruction (CPI):
  - Steps per work item
- Clock frequency:
  - Time for each step

CPU Clocking

- Clock period / Cycle time
- Clock frequency



Computing Performance

$$Clock\ cycles = Instruction\ count \times CPI$$

$$Execution\ time = Clock\ cycles \times Cycle\ time$$

$$= \frac{Clock\ cycles}{Clock\ rate}$$

CPI Example

- Computer A: Cycle time=250ps, CPI=2.0
- Computer B: Cycle time=500ps, CPI=1.2

- Which is faster, and by how much?
  - Hint: Compute how long *one instruction* would take

More on CPI

- Not the same for all instructions!
- We computer an average CPI:

$$Clock\ cycles = \sum_{i=1}^{n} (CPI_i \times Instruction\ count_i)$$

$$CPI = \frac{Clock\ cycles}{Instruction\ count} = \sum_{i=1}^{n} (\frac{CPI_i \times Instruction\ count_i}{Instruction\ count})$$

CPI Example

- Two code sequences containing instructions of classes A, B and C. What is the average CPI?

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

MIPS

- Million Instructions Per Second

$$MIPS = \frac{Instruction\ count}{Execution\ time \times 10^6}$$

- Drawbacks
  - Differences in ISAs between computers
  - Differences in complexity between instructions

Exercise

- What execution times do the following computers have on a program consisting of $5x10^6$ instructions?
- Which has the best performance?

Computer	CPI	Clock frequency
A	2	2.5 GHz
B	0.5	2 GHz
C	1	2.5 GHz

Performance Summary

$$CPU\ Time\ = Instruction\ count\ \times CPI\ \times Cycle\ time$$

- Performance depends on:
  - Algorithm: affects IC, CPI
  - Programming language: affects IC, CPI
  - Compiler: affects IC, CPI
  - ISA: affects IC, CPI and Cycle time

Using Benchmarks

- Measure execution time for representative programs
- The SPEC2006 benchmark suite
  - Standard Performance Evaluation Cooperative
  - Summarize performance using SPECRatio

$$SPECRatio = \frac{Reference\ time}{Execution\ time}$$

Name	Description	IC×10^9	CPI	T_c (ns)	Exec time	Ref time	SPECratio
perl	Interpreted string processing	2,118	0.75	0.4	637	9,777	15.3
bzip2	Block-sorting compression	2,389	0.85	0.4	817	9,650	11.8
gcc	GNU C Compiler	1,050	1.72	0.4	24	8,050	11.1
mcf	Combinatorial optimization	336	10.00	0.4	1,345	9,120	6.8
go	Go game (AI)	1,658	1.09	0.4	721	10,490	14.6
hmmer	Search gene sequence	2,783	0.80	0.4	890	9,330	10.5
sjeng	Chess game (AI)	2,176	0.96	0.4	37	12,100	14.5
libquantum	Quantum computer simulation	1,623	1.61	0.4	1,047	20,720	19.8
h264avc	Video compression	3,102	0.80	0.4	993	22,130	22.3
omnetpp	Discrete event simulation	587	2.94	0.4	690	6,250	9.1
astar	Games/path finding	1,082	1.79	0.4	773	7,020	9.1
xalancbmk	XML parsing	1,058	2.70	0.4	1,143	6,900	6.0
Geometric mean							**11.7**

Summary

This is Bill.

Bill thinks that the higher frequency (GHz) a CPU has, the better its performance is.

Bill will fail CDT204.

Don’t be like Bill.



Summary

01

- Hierarchical layers of abstraction, in both hardware and software
- Cost/performance is improving due to underlying technology development

02

- Instruction Set Architecture (ISA): The hardware/software interface

03

- Use parallelism to improve performance
- Execution time: the best performance measure

Next Time

01

- Instruction set architectures: MIPS, ARM, x86
- Assembly code and machine code

02

- Representing numbers and instructions

03

- From source code to running programs

Do These Today

01

- Read chapter 1 of the course book: All of it, Today!

02

- Read appendix B of the course book: Optional, But good to know

03

- Read the Study Guide carefully, then do Assignment 0

More Stuff

- Watch this if you want to know more about transistors and ICs:
  https://www.youtube.com/watch?v=IcrBqCFLHIY
  https://www.youtube.com/watch?v=7ukDKVHnac4
  https://www.youtube.com/watch?v=aWVywhzuHnQ

The only people who don’t click Skip on ads before YouTube videos, are people who died during that ad.

@Damienfahey